applied sciences

MDPI

Article

A Comparative Study on Self-Driving Scenario Code Generation Through Prompt Engineering Based on LLM-Specific Characteristics

Haneul Yang, Hyoeun Kim and Jonggu Kang *

School of AI Convergence, Sungshin Women's University, Seoul 02844, Republic of Korea; 20221380@sungshin.ac.kr (H.Y.); 20221347@sungshin.ac.kr (H.K.)
* Correspondence: jonggu.kang@sungshin.ac.kr

Abstract

Large Language Models (LLMs) demonstrate potential in code generation capabilities, yet their applicability in autonomous vehicle control has not been sufficiently explored. This study verifies whether LLMs can generate executable MATLAB code for software-defined vehicle scenarios, comparing five models: GPT-4, Gemini 2.5 Pro, Claude Sonnet 4.0, CodeLlama-13B-Instruct, and StarCoder2. Thirteen standardised prompts were applied across three types of scenarios: programming-based driving scenarios, inertial sensor-based simulations, and vehicle parking scenarios. Multiple automated evaluation metrics—BLEU, ROUGE-L, ChrF, Spec-Compliance, and Runtime-Sanity—were used to assess code executability, accuracy, and completeness. The results showed GPT-4 achieved the highest score 0.54 in the parking scenario with an overall average score of 0.27, followed by Gemini 2.5 Pro as 0.26. Commercial models demonstrated over 60% execution success rates across all scenarios, whereas open-source models like CodeLlama and StarCoder2 were limited to under 20%. Furthermore, the parking scenario yielded the lowest average score of 0.19, confirming that complex tasks involving sensor synchronisation and trajectory control represent a common limitation across all models. This study presents a new benchmark for quantitatively evaluating the quality of SDV control code generated by LLMs, empirically demonstrating that prompt design and task complexity critically influence model reliability and real-world applicability.

Keywords: large language model; prompt engineering; MATLAB; software-defined vehicle; code generation; evaluation

check for updates

Academic Editor: Rafael Valencia-Garcia

Received: 22 October 2025
Revised: 8 November 2025
Accepted: 24 November 2025
Published: 25 November 2025

Citation: Yang, H.; Kim, H.; Kang, J. A Comparative Study on Self-Driving Scenario Code Generation Through Prompt Engineering Based on LLM-Specific Characteristics. *Appl. Sci.* **2025**, *15*, 12502. https://doi.org/10.3390/app152312502

Copyright: © 2025 by the authors. Licensee MDPI, Basel, Switzerland. This article is an open access article distributed under the terms and conditions of the Creative Commons Attribution (CC BY) license (https://creativecommons.org/licenses/by/4.0/).

1. Introduction

Software-Defined Vehicles (SDVs) control and improve vehicle functions through software updates and expandability, rather than traditional hardware-centric approaches. This has transformed automobiles into software-centric smart platforms, driven by changes in the industrial environment, such as the widespread adoption of smartphones and the proliferation of electric vehicles [1,2]. SDV requires a massive codebase and continuous feature upgrades. Therefore, a rapid and sustainable verification system is essential to reliably manage short update cycles and large-scale code changes [3].

However, the simulation-based testing methods currently in use have several limitations. The problem of a "gap" has been pointed out, where performance obtained in a virtual environment cannot be generalized to actual road driving [4]. Scenario-based

testing, manually constructed by experts, suffers from a limited scope, failing to adequately encompass rare situations or complex traffic interactions. Furthermore, creating realistic scenarios requires combining diverse variables, including road conditions and surrounding objects, resulting in significant time and expense [5].

To this end, automation and diversity of test cases have emerged as key challenges in SDV verification. Recently, large-scale language models (LLMs) have been utilized in various software engineering tasks based on their code generation capabilities, and the automated generation of autonomous driving simulation code is also attracting attention [6–8]. LLMs specialize in generating simulation code or test scenarios based on natural language input, enabling faster verification of a wider range of situations than traditional manual simulations.

Ye et al. [9] demonstrated that automatically optimizing prompts can improve code generation quality, while Shin et al. [10] revealed that prompt structure directly impacts LLM output performance. Ma et al. [11] proposed LaMPilot, a benchmark for autonomous driving scenario code generation and evaluation, systematically verifying the feasibility and accuracy of LLMs. The LLM4AD project [12] presented the applicability of LLMs in the autonomous driving domain and a simulation-based verification framework.

Previous studies have mostly focused on generating code in general-purpose languages like Python or C, with limited research quantitatively evaluating the executability of generated code or the accuracy of control simulations in MATLAB/Simulink environments. Systematic comparative studies on the effects of prompt engineering strategies tailored to each LLM's characteristics on code quality are also scarce. However, because each model has different performance characteristics, prompt engineering strategies tailored to the specific characteristics of each LLM are required.

Accordingly, this study designed and applied model-specific characteristic-based prompts to five representative LLMs: GPT4, Gemini 2.5 pro, Claude Sonnet 4.0, Code LLaMA 13B-Instruct, and StarCoder2, and compared their code generation performance [3–5]. We compared the runtime and accuracy of the generated code with the automatically generated code by LLM, using the reference code as a baseline for official MATLAB examples of programmatic driving scenarios, synthetic data generation from IMUs, GPS, and wheel encoders, and parking maneuver simulations. The evaluation dichotomously assessed runtime accuracy, and the consistency between the generated and reference codes was quantified using BLEU-4, ROUGE_L_F1, chrF, Token Jaccard, Identifier F1, API F1, and SpecScore, and then comprehensively compared using CompositeScore.

The contributions of this paper are as follows:

- While previous studies merely evaluated code executability in Python-based simulator environments, this study directly executed SDV control code generated by LLMs in the MATLAB/Simulink environment. This enabled verification and evaluation of functional consistency and executability against reference code.
- Unlike previous studies that compared models using a single prompt, this study designed customized prompts considering each LLM's learning characteristics and code generation tendencies, quantitatively comparing their effectiveness.
- It proposes a composite evaluation system integrating practical performance metrics such as executability and specification compliance alongside linguistic similarity indicators, enabling multidimensional code quality assessment and complementing the limitations of existing text-based metrics.

This study systematically compares the executability and quality of LLM-based SDV control code generation in the MATLAB environment. It can serve as foundational data for future research exploring the potential and limitations of LLM utilization in the autonomous driving domain.

This paper is structured as follows. Section 2 analyzes related research, categorizing it into three categories: prompt engineering, code learning and syntax understanding in LLM, and SDV control code and benchmarks. Section 3 presents the methodology of this study and details the experimental design process, including the research questions, prompt engineering and evaluation scale construction, and LLM comparison model. Section 4 synthesizes the experimental results to answer the research questions. Section 5 is a discussion. Section 6 presents the conclusions and proposes directions for future research.

2. Related Work

Ye et al. [13] explored a structure that automatically optimizes prompts themselves, even when simply giving commands to the LLM, utilizing prompt engineering techniques. This process demonstrated that prompt quality directly impacts performance during the complex code generation process. Marvin et al. [14] analyzed the impact of prompt design on response quality to maximize the performance of large-scale language models. Moving beyond simple questions, they conducted experiments based on different prompt types, including role, context, and instructions. This comparison of response quality across large-scale language models highlighted the importance of prompt design and demonstrated the potential of LLMs for solving diverse domain-specific problems. Recently, various benchmark studies have also been proposed to verify the Generalisability of code generation prompt optimization, presenting a framework that comprehensively measures the ability to convert from language to code [15].

Research on code learning and syntax understanding in LLMs is actively underway. Hussain et al. [16] proposed a technique for training large-scale language models to learn command syntax. Using a dedicated tokenizer and learning strategy, the model was able to understand and generate command-based, domain-specific languages. This is crucial for ensuring code quality in domains with strict syntactic structures, such as control systems. Petrovic et al. [17] attempted to automate the automotive software development process by combining model-based engineering with large language models. In this study, they implemented a workflow from requirements to code generation, targeting autonomous braking scenarios within the CARLA simulation environment. Subsequently, they extended the framework to propose an integrated workflow spanning from requirements analysis to control algorithm implementation and vehicle driving. While the proposed framework demonstrated the potential to facilitate automation in software development using generative AI, no comparative analysis using quantitative performance metrics was conducted [18]. Furthermore, recent research has analyzed the degradation of LLM performance in complex code structures by evaluating the accuracy of code generation at the class level, beyond the function level [19]. Such studies underscore the necessity for evaluation methodologies that analyze consistency at both the code structure and identifier levels.

Ali Nouri et al. [20] presented a methodology for automatically verifying and improving autonomous driving control code generated by LLM using a simulation-based feedback loop. Using the ACC case, they compared and evaluated various models, including CodeLlama, DeepSeek, CodeGemma, Mistral, and GPT-4, focusing on improving the quality of LLM-based control code in a simulation environment. Ma et al. [11] proposed a framework for converting natural language commands into executable autonomous driving scenario code based on LLM. They also built LaMPilot, a large-scale benchmark dataset, to evaluate the framework. Specifically, they designed a simulation-based evaluation system to systematically verify the performance of models based on feasibility and accuracy. Additionally, recent research [21] has comprehensively reviewed code generation benchmarks and evaluation metrics, while the CodeJudge framework [22]—which evaluates generated

code without test cases—has been proposed, thereby strengthening the foundation for automated quantitative code quality comparisons.

As summarized in Table 1, we designed prompts to automatically generate executable SDV control algorithm code in MATLAB using various LLMs and compared the resulting code quality. We expanded the two LLMs used in previous studies to three additional LLMs, and quantitatively evaluated code quality according to the prompt design configuration.

Table 1. Comparative analysis of LLM-based code generation studies.

	Main Focus/Domain	Evaluation Metrics	Prompt Design	Model Scope
Ye et al. (2023) [13]	Meta-prompt (PE2) for autonomous prompt refinement	Accuracy improved on reasoning tasks (MultiArith +6.3%, GSM8K +3.1%)	Context-rich meta prompting	Multiple LLMs (no code comparison)
Marvin et al. (2023) [14]	Structured prompts (role, context, instruction)	Human evaluation (no code metric)	Role/context/ instruction mix	GPT-3, BLOOM, OPT
Hussain et al. (2024) [16]	Command-syntax learning for domain-specific languages (DSLs)	Syntax accuracy and recognition rate	Grammar-based fine-tuning	BERT-like model
Petrovic et al. (2024) [17]	Model-driven engineering (MDE) + LLM for automotive (CARLA braking)	Workflow demonstration (no quantitative metrics)	Fixed MDE–LLM pipeline	Single-case study
Ali Nouri et al. (2025) [20]	Simulation-guided code verification (ACC/CAEM scenarios)	Simulation success and stability indicators	Feedback-loop prompts	CodeLlama, GPT-4, etc.
Ma et al. (2024) [11]	*LaMPilot* benchmark for autonomous driving scenarios	Executability and scenario accuracy	Natural-language scenario prompts	Two LLMs in benchmark
Ours	SDV control code generation and execution in MATLAB/Simulink	Executability; BLEU-4; ROUGE-L; chrF; SpecScore	Model-specific prompts for each LLM	Five LLMs (GPT-4, Gemini, Claude, CodeLLaMA, StarCoder2)

3. Methods

Figure 1 illustrates the overall research workflow. Starting from the research question, this study systematically constructed 13 categorized prompts to generate MATLAB (R2025b)-based SDV control code using five representative large language models (LLMs). Subsequently, the code generated by each model was evaluated using eight objective metrics encompassing syntactic similarity, semantic fidelity, and execution validity. Normalized scores were aggregated into a composite score, enabling comparisons between models and analysis of error patterns.

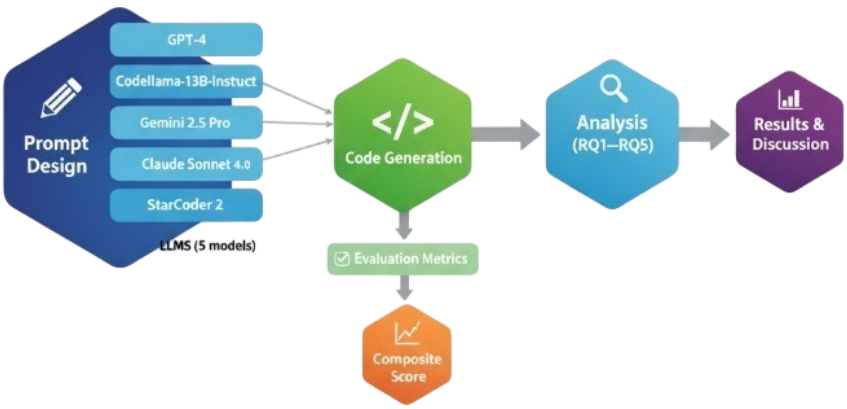

Figure 1. Experimental Workflow of SDV Code Generation.

3.1. Research Questions

- Can LLM automatically generate executable MATLAB code for a given SDV control scenario?
- What differences exist in the accuracy and completeness of the SDV control algorithm code generated by LLM?
- How does the difficulty profile vary across tasks when applying the same prompt design principles?
- How effectively do various automated evaluation metrics and composite scores explain the differences in SDV control code generation performance between LLMs, and what correlations exist between the metrics?
- What were the primary causes of code execution failures, and in which models and tasks did they occur?

3.2. Prompt Engineering

In this study, we utilized a prompt engineering-based LLM to generate MATLAB-based SDV control algorithm code. LLM outputs can vary depending on the model structure, training data, and context processing method, even with identical prompt inputs. Therefore, we designed customized prompts for each model, considering its characteristics and strengths. All models shared a common problem definition, scenario specification, and output format requirements, and code generation was performed under the same conditions. However, we applied differentiated design approaches based on the characteristics and code generation trends of each model. This study designed a total of 13 prompts for practical experiments, broadly categorized into three categories: (1) 5 programmatic driving scenarios, (2) 4 sensor simulations, and (3) 4 parking simulations. This prompt configuration encompassed problems of varying difficulty, from simple trajectory generation to sensor data processing and complex scenario simulations, enabling a multifaceted evaluation of the model's code generation capabilities.

As shown in Table 2, this study differentiated prompts based on model characteristics to maximize the potential of each LLM. Furthermore, by comparing code generation trends across models under identical scenario conditions, this study enabled more realistic and sophisticated performance evaluations than a single-prompt design approach.

Table 2. Prompt Design Characteristics of Each LLM.

Model	Prompt Design Characteristics
GPT-4	Handles complex context and precise code; provided with detailed instructions including specifications
Gemini 2.5 Pro	Strong in numerical tasks; prompt emphasized accuracy and explicit MATLAB API usage
Claude Sonnet 4.0	Good at explanatory and step-by-step reasoning; required code comments and procedural guidance
CodeLlama-13B-Instruct	Limited with long instructions; used concise, direct command-style prompts.
Starcoder2	Code-specialized; prompt included explicit examples and strict output format.

3.3. Performance Metrics

In this study, 9 indicators were used to comprehensively evaluate the quality of MATLAB-based SDV control code generated by LLM. Each metric encompassed code

correctness, syntactic similarity, specification compliance, and executable feasibility, and a composite score was calculated by integrating these metrics.

First, BLEU-4, ROUGE-L, and ChrF are widely used metrics for evaluating syntactic and textual similarity in natural language processing. However, recent studies have reported that these metrics also possess a certain level of validity for basic consistency evaluation of code generation models [23]. Specifically, it has been demonstrated that the BLEU score alone has limitations in judging code generation quality, and when combined with other metrics, it can be used as an auxiliary tool for comparing the syntactic matching or quality of code. Therefore, this study used the bleu-4 evaluation metric in conjunction with the ROUGE-L and ChrF evaluation metrics.

Secondly, the Token Jaccard and Identifier F1 metrics are used as complementary indicators to partially reflect the structural similarity and semantic consistency of the code. Furthermore, it was noted that token-based similarity alone cannot sufficiently capture the logical identity of code, highlighting the necessity for structural comparison based on Abstract Syntax Trees [24]. Building upon this, this study aims to evaluate the semantic consistency and structural coherence of code more precisely, not only through token-level duplication but also by assessing the matching degree at the identifier level.

Thirdly, API Overlap F1 serves as a core metric for evaluating the accuracy and consistency of API calls within code. It has been empirically demonstrated that the presence and accuracy of API calls in LLM-generated code are key determinants of executability and functional suitability [25]. Consequently, this study also incorporates API usage accuracy as an independent quality metric.

Finally, Runtime Sanity serves as an indicator that directly verifies the executability of code. The CodeScore study experimentally demonstrated that evaluations based on code execution better reflect functional consistency and quality than simple text similarity [26]. Therefore, this study also included binary assessment of whether code execution succeeded, quantitatively measuring whether the model generates genuinely functional MATLAB code.

Thus, the evaluation framework of this study is grounded in the theoretical foundations of existing code quality assessment research, comprehensively reflecting syntactic conformity, semantic structural similarity, functional suitability, and executability.

3.3.1. BLEU-4 (Token-Level Precision)

BLEU is a widely used metric for evaluating machine translation performance, calculated based on the 4-gram token matching rate between the generated code and the reference code. Here, P_n denotes n-gram precision, and BP stands for brevity penalty.

$$P_n = \frac{\sum_{g \in G_n(C)} \min(cnt_C(g), cnt_R(g))}{\sum_{g \in G_n} cnt_C(g)} \quad (n = 1, \ldots, 4) \tag{1}$$

$$BP = \begin{cases} 1, & \text{if } |C| > |R| \\ \exp\left(1 - \frac{|R|}{\max(|C|,1)}\right), & \text{otherwise} \end{cases} \tag{2}$$

$$BLEU - 4 = BP \cdot \exp\left(\frac{1}{4} \sum_{n=1}^{4} \log P_n\right) \tag{3}$$

3.3.2. ROUGE-L (Sequence Overlap)

An evaluation metric based on LCS that measures the sequential similarity between generated code and reference code. L denotes the length of the longest common sub-

sequence between two sequences, defining precision P_L and recall R_L. It is ultimately calculated as the harmonic mean of P_L and R_L.

$$P_L = \frac{L}{\max(|C|,1)}, \quad R_L = \frac{L}{\max(|R|,1)}, \quad ROUGE_{L_{F1}} = \frac{2P_L R_L}{P_L + R_L} \tag{4}$$

3.3.3. ChrF (Character-Level F1)

After removing character spaces, $\bar{R}$ and $\bar{C}$ denote the precision/recall rate for each n.

$$P_n = \frac{\sum_{g \in G_n(\overline{C})} \min(cnt_{\overline{C}}(g), cnt_{\overline{R}}(g))}{\sum_{g \in G_n(\overline{C})} cnt_{\overline{C}}(g)}, \tag{5}$$

$$R_n = \frac{\sum_{g \in G_n(\overline{C})} \min(cnt_{\overline{C}}(g), cnt_{\overline{R}}(g))}{\sum_{g \in G_n(\overline{R})} cnt_{\overline{R}}(g)} \tag{6}$$

Similarly to the code implementation, the mean is calculated for n ∈ 2,3, after which F_β is computed.

$$\bar{P} = \frac{P_2 + P_3}{2}, \quad \bar{R} = \frac{R_2 + R_3}{2}, \tag{7}$$

$$chrF = \frac{(1+\beta^2)\bar{P}\bar{R}}{\beta^2\bar{P} + \bar{R}} \quad (\beta = 2) \tag{8}$$

3.3.4. Token Jaccard Similarity

When created using the token sets U_R and U_C of the generated code and reference code, respectively, it is calculated as the ratio of the intersection to the union.

$$Jaccard = \frac{|U_R \cap U_C|}{|U_R \cup U_C|} \tag{9}$$

3.3.5. Identifier F1

After extracting identifiers, it calculates the F1 score for sets I_R and I_C based on whether the identifiers match.

$$TP = |I_R \cap I_C|, \quad FP = |I_C \setminus I_R|, \quad FN = |I_R \setminus I_C| \tag{10}$$

$$P_{id} = \frac{TP}{TP + FP}, \quad R_{id} = \frac{TP}{TP + FN} \tag{11}$$

$$Identifier\ F1 = \frac{2P_{id}R_{id}}{P_{id} + R_{id}} \tag{12}$$

3.3.6. API Overlap F1

For a fixed API list A = plot3, scatter3, . . . , it calculates the F1 score based on the occurrence of each API a ∈ A.

3.3.7. Spec-Compliance Score

It lists the visualization specifications and calculates their weighted sum within the range [0, 100].

3.3.8. Runtime Sanity

It checks if the code executes without syntax errors. If no errors occur, calculate it as 1 in binary; otherwise, calculate it as 0.

3.3.9. Composite Score

Each indicator is normalized to the range [0,1] and integrated as a weighted sum.

3.4. LLM Models

In this study, 5 large-scale language models were selected for comparison to generate autonomous driving code using prompts. In addition to CodeLlama-13B-Instruct and GPT-4, which were used in previous studies, the latest commercial models Gemini 2.5 Pro and Claude Sonnet 4.0, as well as the open-source StarCoder2, were added. GPT-4 [27], a representative high-performance language model developed by OpenAI, demonstrates superior performance in both natural language processing and code generation. It is notably characterized by its proven reliability and wide applicability in real-world industrial environments. CodeLlama-13B-Instruct [28] is based on LLaMA 2, released by Meta, and is optimized for code generation in various programming languages. Its relatively low memory footprint and specialized code generation capabilities based on user instructions make it ideal for efficient use in research environments. Gemini 2.5 Pro [29], a multimodal LLM developed by Google DeepMind, offers enhanced inference capabilities not only in text but also in code generation. In particular, it possesses the performance to handle complex programming tasks based on large parameters and state-of-the-art training datasets. Claude Sonnet 4.0 (https://www.anthropic.com/claude/sonnet (accessed on 22 May 2025), developed by Anthropic, emphasizes safety and intuitive language understanding. It provides high consistency even in large-scale code generation tasks and is particularly strong in user-friendly, interactive code creation. StarCoder2 [30] is an open-source, code-specific language model jointly developed by Hugging Face and ServiceNow Research. It is trained on extensive public code repositories and comprehensively supports various programming languages. Thus, by comparing the various characteristics and strengths of commercial models (GPT-4, Gemini 2.5 Pro, Claude Sonnet 4.0) and open-source models (CodeLlama-13B-Instruct, StarCoder2), we analyzed the performance and applicability of LLM for SDV control code generation.

4. Experimental Results

4.1. RQ1: Can LLM Automatically Generate Executable MATLAB Code for a Given SDV Control Scenario?

Our experimental results showed that all five LLMs generated executable MATLAB code to some degree, but with varying success rates. As illustrated in Figure 2, the execution performance gap among models was substantial. Gemini 2.5 Pro achieved the highest runtime success rate of 53.8% (7 of 13 runs), followed by GPT-4 and Claude Sonnet 4.0, both at 46.2% (6 of 13 runs). In contrast, CodeLlama-13B-Instruct and StarCoder2 failed to execute successfully in any case (0%), underscoring their limited domain alignment with MATLAB syntax and toolboxes. These results clarify that commercially trained LLMs handle MATLAB-specific APIs and runtime dependencies more effectively than open-source code models.

4.2. RQ2: What Differences Exist in the Accuracy and Completeness of the SDV Control Algorithm Code Generated by LLM?

In the experiment, code quality was evaluated based on the Composite Score. According to Table 3, GPT-4 achieved the highest score with an average of 0.276, followed by Gemini 2.5 Pro at 0.263 and Claude Sonnet 4.0 at 0.229. In contrast, StarCoder2 and LLaMA performed significantly worse, with scores of 0.146 and 0.088, respectively. Notably, GPT-4 and Gemini demonstrated stable results not only in execution success rate but also in spec compliance and API call accuracy. GPT-4 achieved an average API utilization score of

0.73 in the driving scenario generation prompt, providing code output that reflected the requirements in the executable code. In contrast, StarCoder2 and LLaMA are optimized for generating language-based code such as Python and C++, and thus significantly reduced code completeness in the MATLAB environment due to typos, array dimension mismatches, and incorrect function calls.

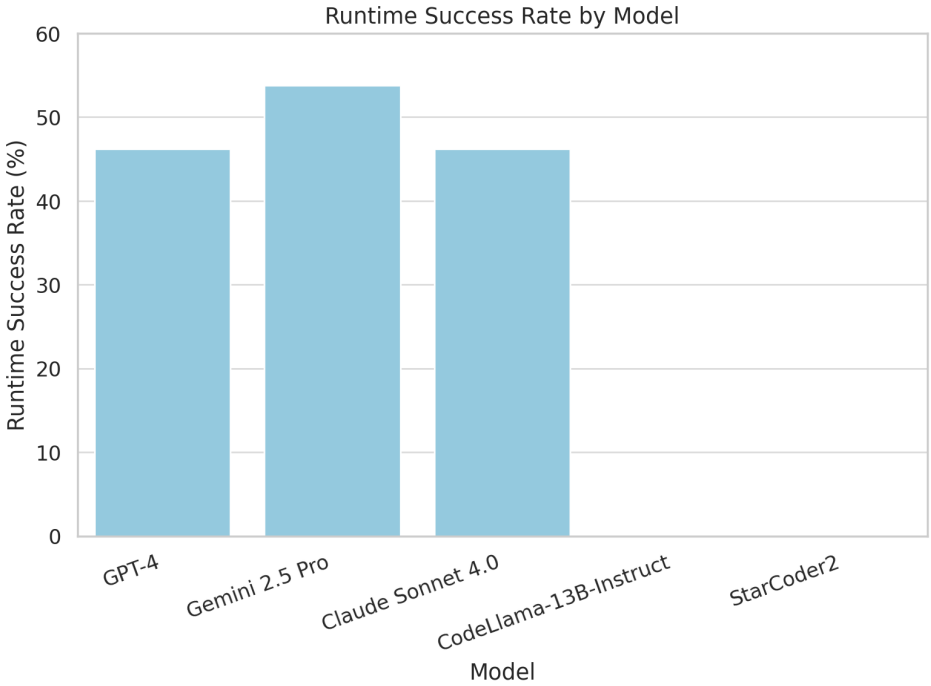

Figure 2. Comparison of Runtime Success Rate by Model.

Table 3. Comparison of Mean Composite Score by Model.

Model	Mean Composite Score
GPT-4	0.276
Gemini 2.5 Pro	0.263
Claude Sonnet 4.0	0.229
CodeLlama-13B-Instruct	0.088
Starcoder2	0.146

4.3. RQ3: How Does the Difficulty Profile Vary Across Tasks When Applying the Same Prompt Design Principles?

When comparing the prompts categorized into three groups—programmatic driving, sensor simulation, and parking simulation—GPT-4 and Gemini exhibited stable results in the first two categories but a sharp decline in the parking scenario in Figure 3. Specifically, GPT-4's mean composite score dropped from 0.29 (programmatic) to 0.18 (parking), while Gemini 2.5 Pro decreased from 0.28 to 0.17 under identical prompt conditions. Claude Sonnet 4.0 maintained moderate performance around 0.22 in simpler tasks but also fell below 0.17 in the parking scenario. CodeLlama and StarCoder2 consistently remained under 0.15 across all task types. These results clearly demonstrate that the parking scenario presents the highest task difficulty, primarily due to its compound requirements—multi-view environment setup, sensor detection and visualization, and strict time-synchronization constraints—which stress the model's reasoning and code-integration capacity.

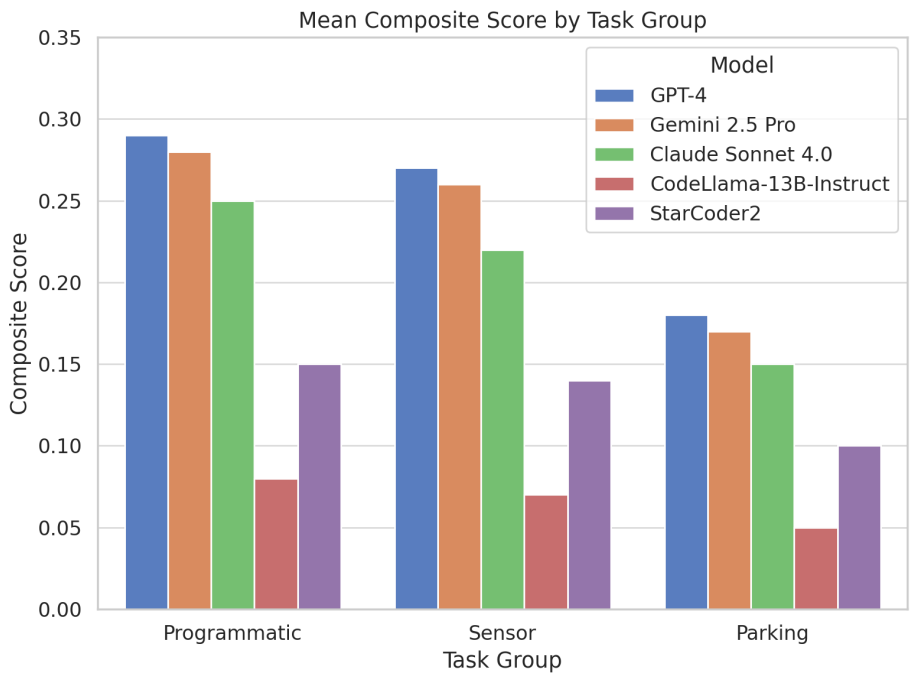

Figure 3. Mean Composite Score by Task Group.

4.4. RQ4: How Effectively Do Various Automated Evaluation Metrics and Composite Scores Explain the Differences in SDV Control Code Generation Performance Between LLMs, and What Correlations Exist Between the Metrics?

Correlation analysis between evaluation metrics was performed on a total of 65 samples. Pearson's correlation coefficient was used to calculate linear relationships between metrics; statistical significance testing or multiple comparison correction was not performed. According to the heatmap results in Figure 4, text-based similarity metrics such as BLEU, ROUGE-L, and chrF showed low correlations (r = 0.2–0.3) with the Composite Score. Conversely, Spec-Compliance and Runtime Sanity exhibited high positive correlations, with r = 0.68 and r = 0.75 respectively. This demonstrates that executability and requirement fulfillment are key factors that better explain the quality of LLM code generation than the degree of syntactic similarity to the correct code. Consequently, future code generation performance evaluations should place greater emphasis on execution-based and specification-based metrics than traditional text-based indicators such as BLEU.

4.5. RQ5: What Were the Primary Causes of Code Execution Failures, and in Which Models and Tasks Did They Occur?

A comprehensive analysis of the failure cases revealed that the primary causes were (1) API signature mismatches, (2) unmet requirements, and (3) array dimension and coordinate system handling errors. Specifically, in the Parking Simulation, GPT-4, Gemini 2.5 Pro, and Claude Sonnet 4.0 repeatedly failed, while CodeLlama-13B-Instruct and StarCoder2 showed overall weaknesses across all tasks. Scatterplot analysis revealed a trend where lower Spec-Compliance scores were also associated with lower Composite Scores. Furthermore, runtime failures are marked with an X in Figure 5, contrasting sharply with success cases marked with an O. This visualization demonstrates that failures are not simply coincidences, but rather stem from clear structural causes (API and specification mismatches).

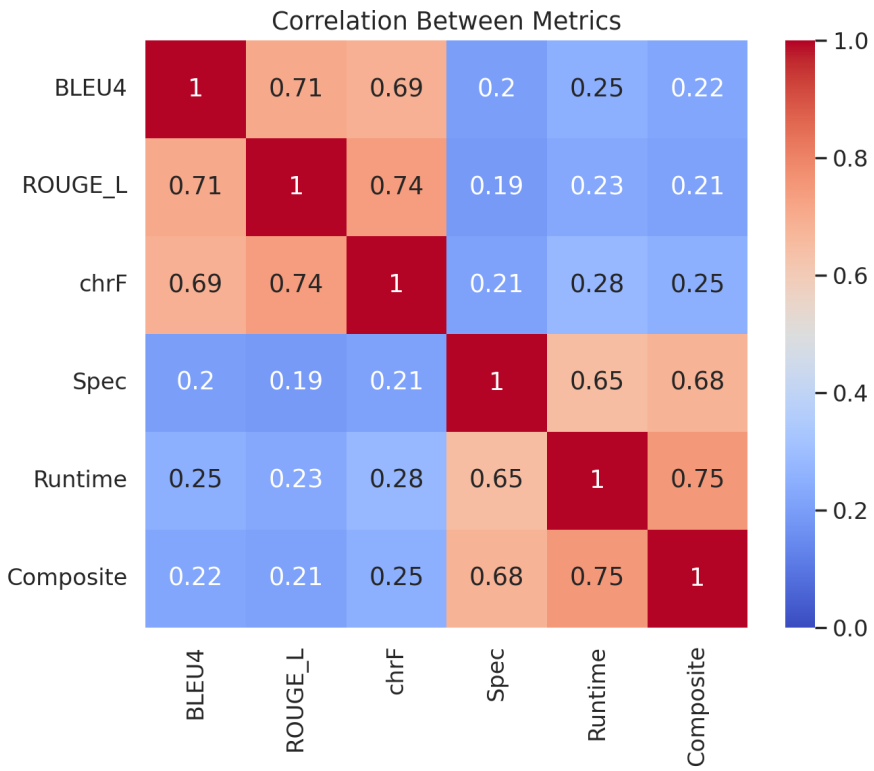

Figure 4. Correlation Heatmap of Evaluation Metrics.

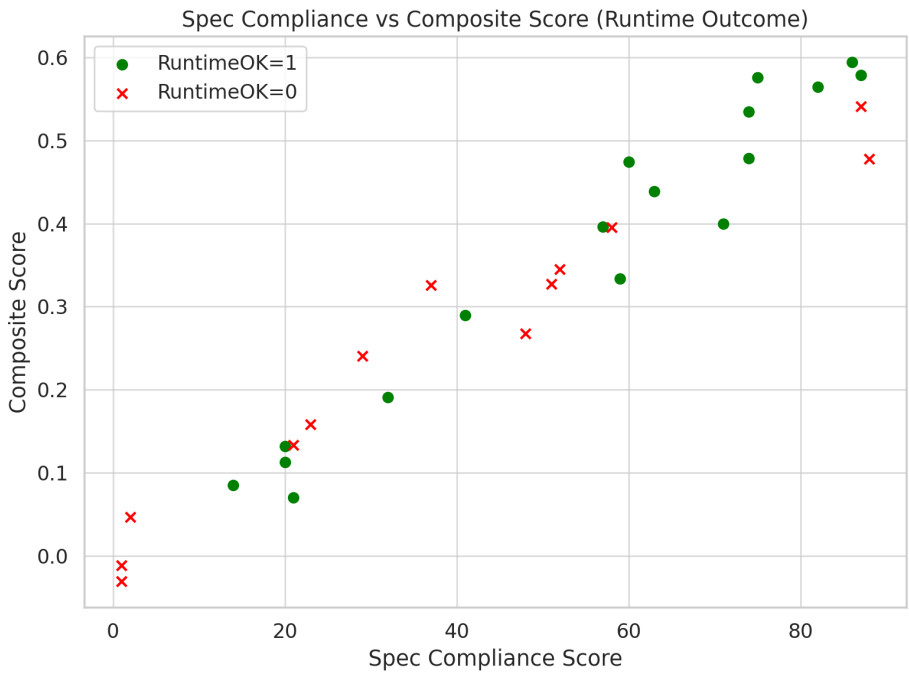

Figure 5. Relationship between Spec Compliance and Composite Score.

5. Discussion

Figures 6–8 show the representative results generated by the GPT-4, Gemini, and Claude models, respectively. All models were trained and executed based on the same scenario, but the main text presents different detailed scenario scenes generated by each model as examples. GPT-4 utilized road boundary recognition, Gemini employed sensor data visualization, and Claude applied parking simulation scenes. Additionally,

the code results consist of MATLAB-based executable code generated by the CodeLlama-13B-Instruct and Starcoder models.

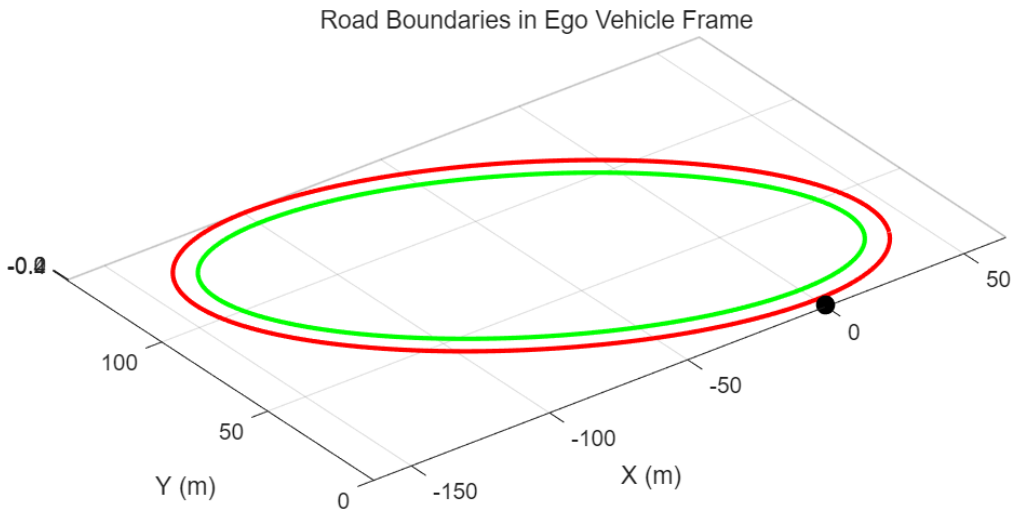

Figure 6. GPT-4 Programmatic Road Boundary Transformation and Ego Coordinate System Results.

GPT-4 demonstrated the highest overall code structural completeness. Notably, in the programming-style example, it fully implemented the road boundary calculation and ego coordinate transformation logic, achieving the best results in chrF (0.45–0.59) and SpecScore (55). This example demonstrates GPT-4's ability to generate structurally consistent code by accurately utilizing mathematical transformations and MATLAB API calls (plot3, cos, sin).

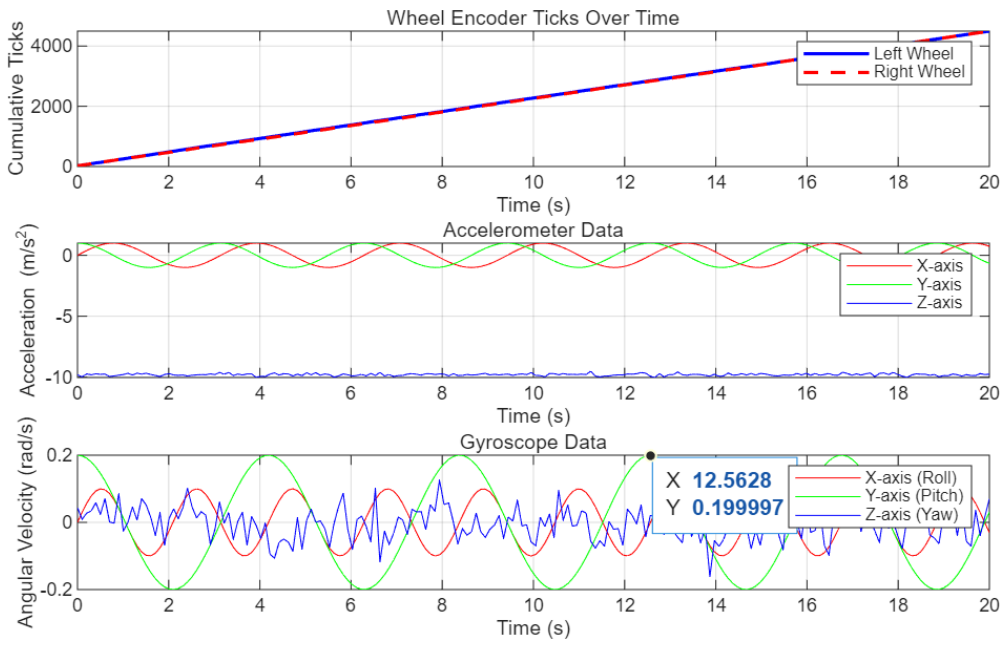

Figure 7. GEMINI SDV Sensor Dashboard and Data Visualization.

Gemini 2.5 Pro exhibited specialized performance in sensor-based simulations. For the SDV sensor example, it generated highly complete code for constructing a dashboard visualizing IMU, wheel encoder, and GPS data. It simultaneously achieved SpecScore (70) and API F1 (1), demonstrating exceptional specification fidelity and graphical representation accuracy. This clearly highlights Gemini 2.5 Pro's strength in numerically intensive, data-processing-centric MATLAB code.

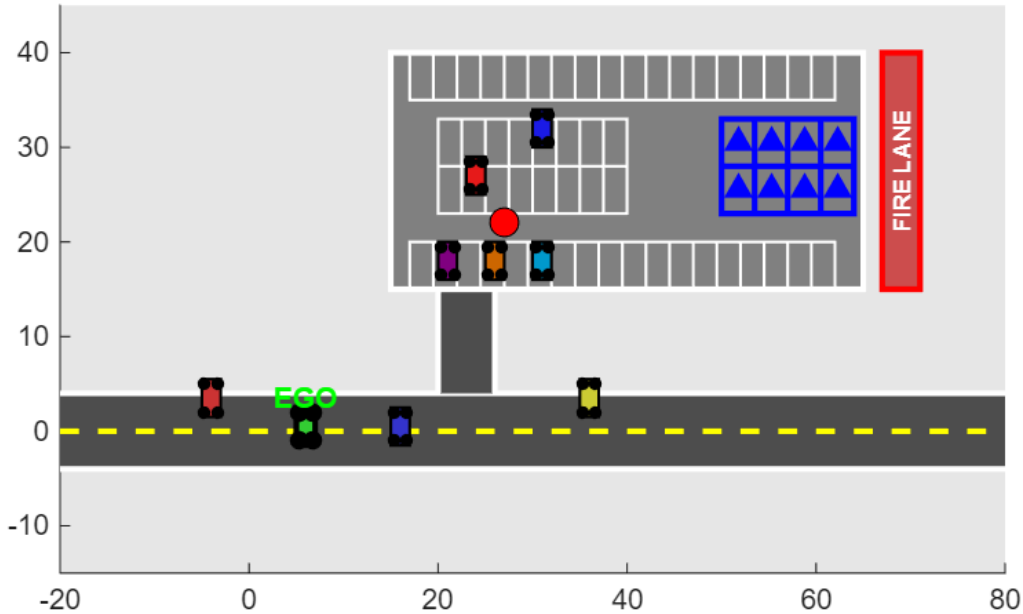

Figure 8. CLAUDE 3-View Parking Simulation.

Claude Sonnet 4.0 excelled most in code readability and comment structure. For Parking Example 1, it visualised the vehicle's parking trajectory while outputting each stage's status in log format. Although its execution stability was lower than GPT-4 or Gemini, it received high marks for logical step-by-step explanations and visual completeness. This demonstrates Claude's balanced reflection of linguistic reasoning and structural explanatory capabilities during code generation.

CodeLlama-13B-Instruct exhibited the highest incidence of MATLAB syntax recognition errors. Programming Example 3 resulted in an 'Invalid expression' error during execution, revealing limitations in parsing and indexing processing. This indicates that while Llama excels at learning general-purpose languages like Python and C++, it is not optimised for MATLAB's vector operations and function call syntax.

StarCoder2 exhibited unstable code execution but showed consistent potential in learning code structure. In the parking example results, "g" undefined and "car model" errors occurred, yet the graphical elements and simulation flow itself were partially reproduced. This suggests StarCoder2 exhibits a tendency to learn code structure but lacked prior training on MATLAB's syntax and API call systems.

The above results demonstrate that the code generation characteristics of each model are distinctly different. GPT-4 and Gemini 2.5 Pro showed strengths in specification fidelity and execution stability, while Claude Sonnet 4.0 excelled in readability and narrative structure. CodeLlama-13B-Instruct and StarCoder2, as open-source models, revealed limitations in domain adaptation within the MATLAB domain. This comparison empirically demonstrates that performance analysis of LLM-based SDV control code generation requires evaluation centred on execution and specification fulfilment, rather than syntactic similarity.

6. Conclusions and Future Work

This study compared the performance of five representative LLMs (GPT-4, Gemini 2.5 Pro, Claude Sonnet 4.0, StarCoder2, and CodeLlama-13B-Instruct) in automatically generating MATLAB code related to autonomous vehicle control scenarios. A total of 13 prompts were designed and nine evaluation metrics were applied. While some models generated executable code within a limited scope, their overall execution success rates remained below 55%. GPT-4 and Gemini 2.5 Pro demonstrated superior performance, with relatively high overall scores and execution success rates. Claude demonstrated moderate perfor-

mance, while StarCoder and LLaMA demonstrated weaknesses in executable performance. While the Programmatic and sensor simulation prompt types yielded acceptable results, all models consistently underperformed on complex tasks requiring multiple views, sensor detection, and time synchronization, such as parking simulation. Metric analysis revealed that text similarity metrics do not adequately explain code executable performance. Instead, execution and requirement-satisfaction metrics, such as Spec Compliance, API Overlap F1, and Runtime Sanity, were more effective in explaining performance differences. Failures were primarily categorized as API signature and parameter mismatches, non-compliance with requirements, and coordinate system and dimension errors. These findings suggest that LLM code generation evaluations should place greater emphasis on executable performance and specification compliance, rather than relying solely on syntactic similarity. The prompts and evaluation scripts used in this study can be provided by the corresponding author upon request to ensure the reproducibility of the research. This is expected to enable subsequent researchers to verify the results in the same experimental environment and expand upon the prompt design strategies.

Future research should comprehensively evaluate LLM's code generation capabilities, including a wider range of autonomous driving scenarios. Specifically, we plan to expand the dataset for tasks that underperformed in this study and explore prompt optimization strategies to improve performance. Furthermore, by comparing model performance across autonomous driving software environments, such as Python, C++, and ROS, we will ensure compatibility with various development frameworks and enhance the reliability of simulation verification results.

Author Contributions: Conceptualization, H.Y. and H.K.; methodology, H.Y. and H.K.; software, H.Y. and H.K.; validation, H.Y. and H.K.; formal analysis, H.Y. and H.K.; investigation, H.Y. and H.K.; resources, J.K.; data curation, H.Y. and H.K.; writing—original draft preparation, H.Y. and H.K.; writing—review and editing, H.Y. and H.K.; visualization, H.Y. and H.K.; supervision, J.K.; project administration, J.K.; funding acquisition, J.K. All authors have read and agreed to the published version of the manuscript.

Funding: This work was supported by the Sungshin Women's University Research Grant of 2025.

Institutional Review Board Statement: Not applicable.

Informed Consent Statement: Not applicable.

Data Availability Statement: The original contributions presented in this study are included in the article. Further inquiries can be directed to the corresponding author.

Conflicts of Interest: The authors declare no conflicts of interest.

Abbreviations

The following abbreviations are used in this manuscript:

LLM	Large Language Model
SDV	Software Defined Vehicle
API	Application Programming Interface

References

1. Liu, Z.; Zhang, W.; Zhao, F. Impact, challenges and prospect of software-defined vehicles. *Automot. Innov.* **2022**, *5*, 180–194. [CrossRef]
2. Kang, J. Software Practice and Experience on Smart Mobility Digital Twin in Transportation and Automotive Industry: Toward SDV-Empowered Digital Twin Through EV Edge-Cloud and AutoML. *J. Web Eng.* **2024**, *23*, 1155–1180. [CrossRef]

3. Bhattacharjee, A.; Mahmood, H.; Lu, S.; Ammar, N.; Ganlath, A.; Shi, W. Edge-assisted over-the-air software updates. In Proceedings of the 2023 IEEE 9th International Conference on Collaboration and Internet Computing (CIC), Atlanta, GA, USA, 1–4 November 2023; IEEE: New York, NY, USA, 2023; pp. 18–27.
4. Menzel, T.; Bagschik, G.; Maurer, M. Scenarios for development, test and validation of automated vehicles. In Proceedings of the 2018 IEEE Intelligent Vehicles Symposium (IV), Suzhou, China, 26–30 June 2018; IEEE: New York, NY, USA, 2018; pp. 1821–1827.
5. Fremont, D.J.; Kim, E.; Pant, Y.V.; Seshia, S.A.; Acharya, A.; Bruso, X.; Wells, P.; Lemke, S.; Lu, Q.; Mehta, S. Formal scenario-based testing of autonomous vehicles: From simulation to the real world. In Proceedings of the 2020 IEEE 23rd International Conference on Intelligent Transportation Systems (ITSC), Rhodes, Greece, 20–23 September 2020; IEEE: New York, NY, USA, 2020; pp. 1–8.
6. Koziolek, H.; Grüner, S.; Hark, R.; Ashiwal, V.; Linsbauer, S.; Eskandani, N. LLM-based and Retrieval-Augmented Control Code Generation. In Proceedings of the 1st International Workshop on Large Language Models for Code (LLM4Code'24), Lisbon, Portugal, 20 April 2024.
7. Aasi, E.; Nguyen, P.; Sreeram, S.; Rosman, G.; Karaman, S.; Rus, D. Generating Out-Of-Distribution Scenarios Using Language Models. *arXiv* **2024**, arXiv:2411.16554. [CrossRef]
8. Zhang, J.; Xu, C.; Li, B. ChatScene: Knowledge-Enabled Safety-Critical Scenario Generation for Autonomous Vehicles. In Proceedings of the 2024 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR), Seattle, WA, USA, 16–22 June 2024; pp. 15459–15469. [CrossRef]
9. Ye, S.; Sun, Z.; Wang, G.; Guo, L.; Liang, Q.; Li, Z.; Liu, Y. Prompt Alchemy: Automatic Prompt Refinement for Enhancing Code Generation. *arXiv* **2025**, arXiv:2503.11085. [CrossRef]
10. Shin, J.; Tang, C.; Mohati, T.; Nayebi, M.; Wang, S.; Hemmati, H. Prompt Engineering or Fine-Tuning: An Empirical Assessment of LLMs for Code. *arXiv* **2025**, arXiv:2310.10508.
11. Ma, Y.; Cui, C.; Cao, X.; Ye, W.; Liu, P.; Lu, J.; Abdelraouf, A.; Gupta, R.; Han, K.; Bera, A.; et al. Lampilot: An open benchmark dataset for autonomous driving with language model programs. In Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition, Seattle, WA, USA, 16–22 June 2024; pp. 15141–15151.
12. Cui, C.; Ma, Y.; Yang, Z.; Zhou, Y.; Liu, P.; Lu, J.; Li, L.; Chen, Y.; Panchal, J.H.; Abdelraouf, A.; et al. Large Language Models for Autonomous Driving (LLM4AD): Concept, Benchmark, Experiments, and Challenges. *arXiv* **2024**, arXiv:2410.15281.
13. Ye, Q.; Axmed, M.; Pryzant, R.; Khani, F. Prompt engineering a prompt engineer. *arXiv* **2023**, arXiv:2311.05661.
14. Marvin, G.; Hellen, N.; Jjingo, D.; Nakatumba-Nabende, J. Prompt engineering in large language models. In Proceedings of the International Conference on data Intelligence and Cognitive Informatics, Tirunelveli, India, 27–28 June 2023; Springer: Berlin/Heidelberg, Germany, 2023; pp. 387–402.
15. Ni, A.; Yin, P.; Zhao, Y.; Riddell, M.; Feng, T.; Shen, R.; Cohan, A. L2CEval: Evaluating language-to-code generation capabilities of large language models. *Trans. Assoc. Comput. Linguist.* **2024**, *12*, 1311–1329. [CrossRef]
16. Hussain, Z.; Nurminen, J.K.; Ranta-aho, P. Training a language model to learn the syntax of commands. *Array* **2024**, *23*, 100355. [CrossRef]
17. Petrovic, N.; Pan, F.; Lebioda, K.; Zolfaghari, V.; Kirchner, S.; Purschke, N.; Khan, M.A.; Vorobev, V.; Knoll, A. Synergy of large language model and model driven engineering for automated development of centralized vehicular systems. *arXiv* **2024**, arXiv:2404.05508. [CrossRef]
18. Petrovic, N.; Pan, F.; Zolfaghari, V.; Lebioda, K.; Schamschurko, A.; Knoll, A. GenAI for Automotive Software Development: From Requirements to Wheels. *arXiv* **2025**, arXiv:2507.18223. [CrossRef]
19. Du, X.; Liu, M.; Wang, K.; Wang, H.; Liu, J.; Chen, Y.; Lou, Y. Evaluating large language models in class-level code generation. In Proceedings of the IEEE/ACM 46th International Conference on Software Engineering (ICSE), Lisbon, Portugal, 14–20 April 2024; IEEE: New York, NY, USA, 2024; pp. 1–13.
20. Nouri, A.; Andersson, J.; Hornig, K.D.J.; Fei, Z.; Knabe, E.; Sivencrona, H.; Cabrero-Daniel, B.; Berger, C. On Simulation-Guided LLM-based Code Generation for Safe Autonomous Driving Software. *arXiv* **2025**, arXiv:2504.02141.
21. Paul, D.G.; Zhu, H.; Bayley, I. Benchmarks and metrics for evaluations of code generation: A critical review. In Proceedings of the 2024 IEEE International Conference on Artificial Intelligence Testing (AITest), Shanghai, China, 15–18 July 2024; IEEE: New York, NY, USA, 2024, pp. 87–94.
22. Tong, W.; Zhang, T. CodeJudge: Evaluating code generation with large language models. *arXiv* **2024**, arXiv:2410.02184. [CrossRef]
23. Evtikhiev, M.; Bogomolov, E.; Sokolov, Y.; Bryksin, T. Out of the BLEU: How should we assess quality of the code generation models? *J. Syst. Softw.* **2023**, *203*, 111741. [CrossRef]
24. Song, Y.; Lothritz, C.; Tang, D.; Bissyandé, T.F.; Klein, J. Revisiting code similarity evaluation with abstract syntax tree edit distance. *arXiv* **2024**, arXiv:2404.08817. [CrossRef]
25. Wu, Y.; He, P.; Wang, Z.; Wang, S.; Tian, Y.; Chen, T.H. A comprehensive framework for evaluating API-oriented code generation in large language models. *arXiv* **2024**, arXiv:2409.15228. [CrossRef]
26. Dong, Y.; Ding, J.; Jiang, X.; Li, G.; Li, Z.; Jin, Z. CodeScore: Evaluating code generation by learning code execution. *ACM Trans. Softw. Eng. Methodol.* **2025**, *34*, 1–22. [CrossRef]

27. Ságodi, Z.; Antal, G.; Bogenfürst, B.; Isztin, M.; Hegedűs, P.; Ferenc, R. Reality check: Assessing GPT-4 in fixing real-world software vulnerabilities. In Proceedings of the 28th International Conference on Evaluation and Assessment in Software Engineering, Salerno, Italy, 18–21 June 2024; pp. 252–261.
28. Roziere, B.; Gehring, J.; Gloeckle, F.; Sootla, S.; Gat, I.; Tan, X.E.; Adi, Y.; Liu, J.; Sauvestre, R.; Remez, T.; et al. Code llama: Open foundation models for code. *arXiv* **2023**, arXiv:2308.12950.
29. Comanici, G.; Bieber, E.; Schaekermann, M.; Pasupat, I.; Sachdeva, N.; Dhillon, I.; Blistein, M.; Ram, O.; Zhang, D.; Rosen, E.; et al. Gemini 2.5: Pushing the frontier with advanced reasoning, multimodality, long context, and next generation agentic capabilities. *arXiv* **2025**, arXiv:2507.06261. [CrossRef]
30. Li, R.; Allal, L.B.; Zi, Y.; Muennighoff, N.; Kocetkov, D.; Mou, C.; Marone, M.; Akiki, C.; Li, J.; Chim, J.; et al. Starcoder: May the source be with you! *arXiv* **2023**, arXiv:2305.06161. [CrossRef]

Disclaimer/Publisher's Note: The statements, opinions and data contained in all publications are solely those of the individual author(s) and contributor(s) and not of MDPI and/or the editor(s). MDPI and/or the editor(s) disclaim responsibility for any injury to people or property resulting from any ideas, methods, instructions or products referred to in the content.

Reproduced with permission of copyright owner. Further reproduction prohibited without permission.